Supplementary Materials for

Multiple machine learning approach to characterize two-dimensional nanoelectronic devices via featurization of charge fluctuation

Kookjin Lee[1,2,3‡], Sangjin Nam[4‡], Hyunjin Ji[5], Junhee Choi[6], Jun-Eon Jin[7], Yeonsu Kim[3], Junhong Na[8], Min-Yeul Ryu[8], Young-Hoon Cho[7], Hyebin Lee[7], Jaewoo Lee[3], Min-Kyu Joo[9,10], and Gyu-Tae Kim[3]**

[1]imec, Leuven, Belgium

[2]Department of Materials Science, KU Leuven, Leuven, Belgium

[3]School of Electrical Engineering, Korea University, Seoul, Republic of Korea

[4]Department of Computer Science and Engineering, Korea University, Seoul, Republic of Korea

[5]School of Electrical Engineering, University of Ulsan, Republic of Korea

[6]Defense agency for technology and quality, Jinju-si, Gyeongsangnam-do, Republic of Korea

[7]Samsung Electronics Co. Ltd, 1 Samsung-ro, Yongin-si, Gyeonggi-do, Republic of Korea

[8]Samsung Display Co. Ltd, 1 Samsung-ro, Yongin-si, Gyeonggi-do 17113, Republic of Korea

[9]Department of Applied Physics, Sookmyung Women's University, Seoul, Republic of Korea

[10]Institute of Advanced Materials and Systems, Sookmyung Women's University, Seoul 04310, Republic of Korea

[*]e-mail: mkjoo@sookmyung.ac.kr & gtkim@korea.ac.kr

‡These authors contributed equally.

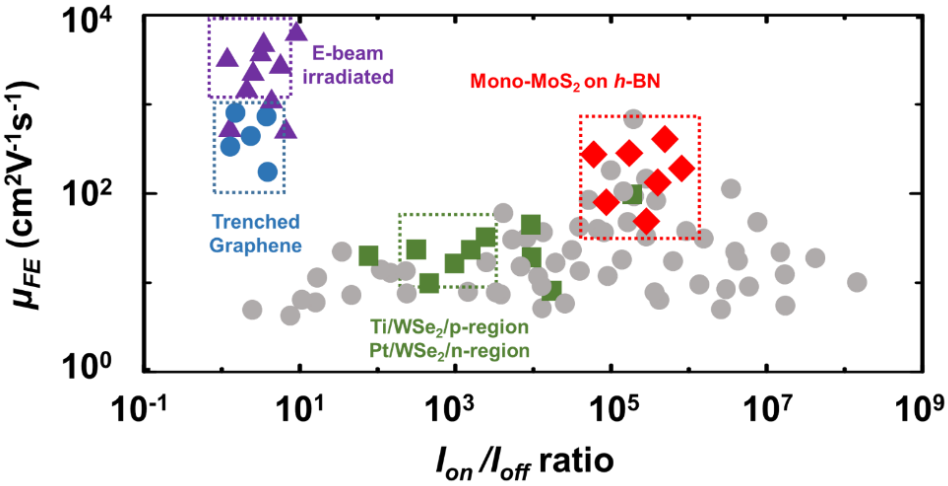
This file includes:

Supplementary Notes 1-5

References

Supplementary Notes

Supplementary Note 1: Types of 2D layered FETs obtained through various combinations of conditions



Supplementary Figure 1. Various 2D layered field-effect transistors (FETs) fabricated under different conditions and external factors: Comparison between the field-effect mobility and current on/off ratio of the 2D layered FETs under various conditions.

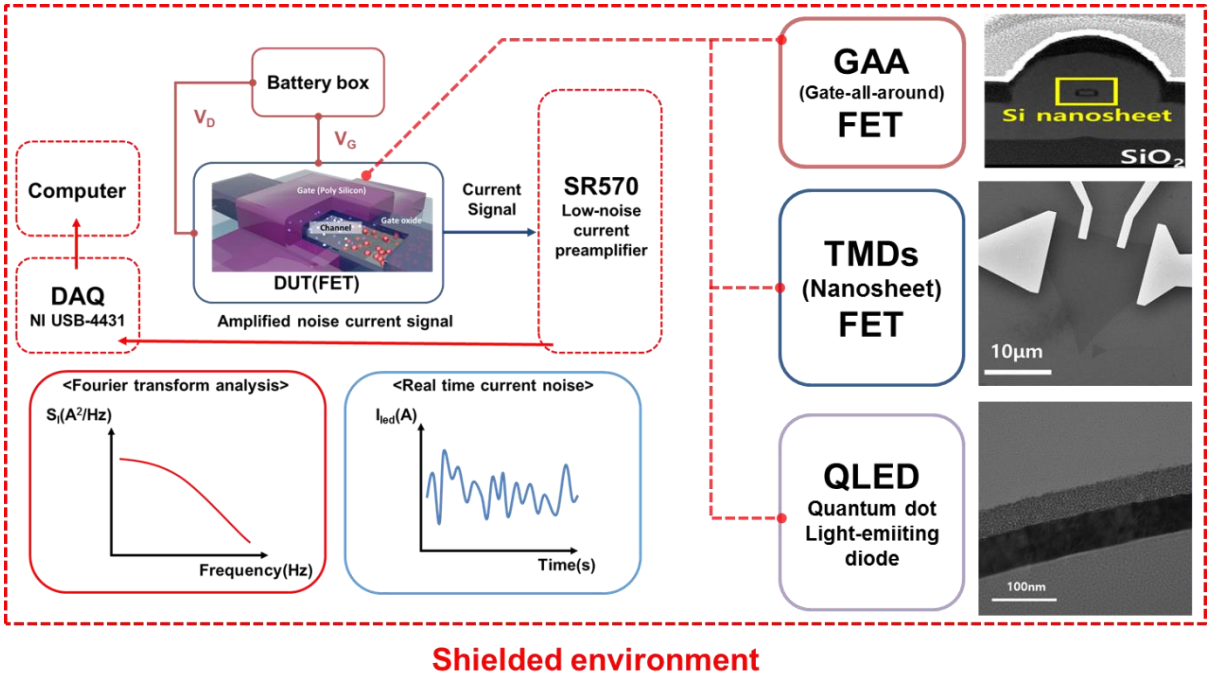
The 2D layered FETs, which have been fabricated and analyzed under various conditions, such as different gate dielectrics (SiO_2 or *h*-BN),[1–5] external factors (e-beam irradiation, TEOA doping, or measurement temperature variation),[1,2,5–7] channel materials (MoS_2, $MoTe_2$, WSe_2, ReS_2, graphene, and BP),[1–4,8–10] and source/drain contact metals (Au, Ti, Pt or Cr),[1,2] for decades are illustrated in Supplementary Figure 1. In total, 103 different 2D layered FETs were analyzed electrically under 32 different conditions using various gate and drain voltages, and their low-frequency noise was measured. Supplementary Figure 1 shows the field-effect mobility (μ_{FE}) versus current on/off ratio (I_{on}/I_{off}) extracted from the transfer characteristics (I_D-V_G) of the 2D layered FETs under various conditions. MoS_2[2,7,8,11] and ReS_2[12] FETs exhibited *n*-type characteristics, whereas $MoTe_2$,[1,13] WSe_2,[4,14–16] and BP[3] FETs exhibited *n*- or *p*-type dominant ambipolar characteristics, depending on the source/drain contact metal. The graphene[9] FETs exhibited relatively high μ_{FE}, as expected for conductors, but exhibited low I_{on}/I_{off}. Moreover, the 2D layered FETs irradiated with the e-beam also exhibited characteristics similar to those of the conductors. The graphene and e-beam irradiated FETs exhibited characteristics similar to those of the conductors;[9] thus, these characteristics could be classified easily using fundamental electrical analyses such as μ_{FE} and I_{on}/I_{off}. However, as most of the 2D layered FETs also exhibited fundamental electrical properties such as μ_{FE}, I_{on}/I_{off}, and subthreshold swing over a wide range, they could not be classified easily.

Supplementary Note 2: Low-Frequency Noise Theory

2.1 Homebuilt low-frequency noise measurement set-up

The low-frequency noise in devices is analyzed as the presence of traps, defects, and lattice damage in targeted devices. Accurately measuring and characterizing substantially small and sensitive noise signals from the device is very challenging, and a well-designed measurement system is required. The measurement system consists of several electrical components; a power source, a pre-amplifier, and other parts for data acquisition or spectrum analysis. The signal or data obtained from the measurement should be the results of the targeted device only and not be affected by external signals such as RF or other electrical signals. Therefore, the noise measurement system must be designed using appropriate shielding and preferably power sources with batteries to avoid external noises. Using batteries as a power source for the device can eliminate external noises introduced into the device. Alternatively, using a filtered voltage source achieves adequate stability and also enables automatic measurements.

A homebuilt system was designed using a battery box, a low-noise current pre-amplifier (SR570), and a data acquisition system (DAQ). The current fluctuation signal of device-under-test (DUT) passes through the low-noise current pre-amplifier and is amplified. In addition, a low-noise current pre-amplifier converts the current signal into a voltage signal, which is transmitted to a spectrum analyzer through the DAQ.[17] A spectrum analyzer uses the Fast Fourier transform (FFT) algorithm to convert the current signal in the time domain into power spectral density (PSD) in the frequency domain.



Supplementary Figure 2. Homebuilt low-frequency noise system for various nanoelectric devices

2.2 McWorther Model

This theory explains the origin of number fluctuations caused by the tunneling transition of traps and carriers. It is based on the well-known theory of A. L. McWorther, which provides a basic understanding of 1/*f* noise generated in MOSFETs.[18–20] The generation-recombination (g-r) noise caused by electrons being randomly captured and released by one trap in the gate dielectric, $S_{Q_{ox}}$, is expressed as follows:

$$S_{Q_{ox}} = \frac{q^2}{W^2L^2} 4\overline{\Delta N_{ox}{}^2} \frac{\tau}{1+(2\pi f\tau)^2} \quad , \tag{1}$$

where q is the electron charge, W and L are the width and length of the channel, τ is the time constant of the transient, N_{ox} is the number of oxide charges generated by one trap with the energy of E, and $\overline{\Delta N_{ox}{}^2}$ is the variance of the probability of oxide charges and expressed as follows.

$$\overline{\Delta N_{ox}{}^2} = f(E)\big(1 - f(E)\big), f(E) = \left(1 + e^{\frac{E-E_F}{kT}}\right)^{-1} \tag{2}$$

The variance of the number of oxide charges is calculated from the Fermi-Dirac distribution function, $f(E)$. Thus, to confirm the contributions of all traps present in the gate dielectric to the power spectral density of noise in volume and energy, we assume a trap density, N_t, in volume and energy and make an integration Eq. (1) and (2) as follows.

$$S_{Q_{ox}} = \frac{q^2}{W^2L^2} \int_{E_V}^{E_c} \int_0^W \int_0^L \int_0^{t_{ox}} 4N_t f(E)\big(1 - f(E)\big) \frac{\tau}{1+(2\pi f\tau)^2} dxdydzdE \tag{3}$$

$$S_{Q_{ox}} = \frac{q^2kT}{WL} \int_0^{t_{ox}} 4N_t \frac{\tau}{1+(2\pi f\tau)^2} dz \tag{4}$$

where T is the temperature, and t_{ox} is the thickness of the oxide dielectric, E_c is the conduction band energy level, E_v is the valance band energy level, and N_t is the density of traps in the gate dielectric at the quasi-Fermi level.

In the McWorther model, we suppose that the capture and release processes occur through tunneling, and the trapping time constant is expressed as follows:

$$\tau = \tau_0(E)e^{\frac{z}{\lambda}} \quad , \tag{5}$$

where z is the location of trap at a certain distance in the gate dielectric and λ is the tunneling attenuation length, which is predicted using the Wentzel-Kramers-Brillouin theory and is expressed as follows.

$$\lambda = \left[\frac{4\pi}{h}\sqrt{2m^*\Phi_B}\right]^{-1} \quad , \tag{6}$$

where Φ_B is the tunneling barrier height obtained using the carriers at the interface between the channel and gate dielectric, and h is the Planck's constant. By combining Eq. (4) and (5), we obtain the bias dependence of the normalized drain current noise in the carrier number fluctuations, S_V:

$$S_V = \frac{q^2 kT\lambda N_t}{f^{\gamma} WLC_{ox}{}^2} \tag{7}$$

where γ is the frequency exponent, which is indicates the location of trap density.

2.3 Correlated Mobility Fluctuations Model

The trapped carrier, explained by the carrier number fluctuations of McWorther model, shifts the flat-band voltage, thereby causing fluctuation in the inversion charge density, which consequently affects the mobility.[18–20] This mobility fluctuation is related to the fluctuation in the inversion charge density as well as the number fluctuations caused by the capture and release of carriers. The effect of such correlated mobility fluctuations on the model is usually expressed as the scattering parameter (α_{SC}), which is mainly estimated by characterizing the low-frequency noise and mobility. This parameter can be expressed as follows:[20]

$$\alpha_{SC} = \frac{1}{\mu_{eff}{}^2}\frac{\partial \mu_{eff}}{\partial Q_{ox}} = \frac{1}{\mu_{ac}{}^2}\frac{\partial \mu_{ac}}{\partial Q_i} + \frac{1}{\mu_{sr}{}^2}\frac{\partial \mu_{sr}}{\partial Q_i} + \frac{1}{\mu_{C,imp}{}^2}\frac{\partial \mu_{C,imp}}{\partial Q_i} + \frac{1}{\mu_{C,ox}{}^2}\frac{\partial \mu_{C,ox}}{\partial Q_i} \ , \tag{8}$$

where μ_{eff} is the channel effective mobility, μ_{ac} is the mobility limited by surface acoustic phonon scattering, μ_{sr} denotes the mobility caused by surface roughness scattering, $\mu_{C,imp}$ represents the mobility caused by Coulomb scattering occurring mainly due to ionized impurities, and $\mu_{C,ox}$ is the mobility limited by Coulomb scattering occurring mainly due to the fixed/trapped charge in the gate oxide.

If we convert the above equations into the current noise equation, we will obtain the following equation:[18–20]

$$S_V = \frac{S_I}{g_m{}^2} = \frac{q^2 kT\lambda N_t}{f^{\gamma} WLC_{ox}{}^2}\left(1 + \frac{\alpha_{SC}\mu_{eff}C_{ox}I_D}{g_m}\right)^2 \tag{9}$$

As a result, the power spectral densities, which are generally obtained using the Fourier transform, can be expressed by Eq. (9). Generally, we can eliminate the scattering parameter α_{SC} and express it by adding N_t^{μ} instead of N_t. N_t^{μ} represents the concentration of scattering centers of a particular type that contribute to the noise.

Regardless of how it is expressed, the resulting power spectral density is affected by the FET's dimensions, gate dielectric parameters, frequency, trap density, and scattering parameters. Moreover, it is observed that the power spectral density is significantly affected by the frequency, trap density, and scattering parameters, when they vary more than 10 times.

Supplementary Note 3: Machine Learning and Deep Learning

3.1 Classification

Supervised learning learns the relationship between the functions that occur in a data set (x, y), where input x and output y are paired.[21] Therefore, if the output value y is a continuous value, the process is regression, whereas a discrete output value indicates that the process is classification. In ML, the classification is the process of predicting a label of specified data sets.

3.2 Bayes Optimal Classifier

For a given input x, the probability of belonging to a certain class is calculated using Bayes' Rule as follows:[22]

$$\mathrm{P}(\mathrm{C}|x) = \frac{P(C)P(x|C)}{P(x)} \tag{10}$$

If there are K classes C_1, C_2, …, C_K, then the probability of a given input x belonging to class C_i is calculated as follows:

$$\mathrm{P}(C_i|x) = \frac{P(C_i)P(x|C_i)}{P(x)} = \frac{P(C_i)P(x|C_i)}{\sum_{k=1}^{K} P(x|C_k)P(C_k)} \tag{11}$$

Thus, the Bayes optimal classifier is provided by the following equation:

$$\text{Choose } \mathrm{argmax}_{C_k} P(C_k|x) \tag{12}$$

3.3 Single-Layer Perceptron (SLP)

Perceptron, which was proposed in 1957 by Resenblatt, is a machine learning model that acts as a linear classifier.[21,23] It is assumed that this model works by calculating the sum of weights of the input layer first and then performing a non-linear transformation on the results because it consists of only one layer. The sum of the products of input node value x_i and weight w_i is calculated using the following equation:

$$\hat{y} = \sum_{i=1}^{d} w_i x_i + w_0 = \sum_{i=0}^{d} w_i x_i \text{ where } x_0 = 1 \tag{13}$$

The calculated results are then passed through a nonlinear activation function. The activation function shown as the following equation was originally proposed by Resenblatt as a step function, however it uses several sigmoid functions.

$$y = f(\hat{y}) = \frac{1}{1+e^{-\hat{y}}} \tag{14}$$

In contrast, SLP creates a line that corresponds to the interface in the input space, facilitating the categorization of two classes. Thus, to categorize multiple classes, an SLP with multiple output nodes, where each output is calculated using the following formula, was composed:

$$y_k = f(\widehat{y_k}) = \frac{1}{1+e^{-\sum_{i=0}^{d} w_{ki} x_i}} \tag{15}$$

3.4 Activation Function

The activation function is neurologically considered as a process of firing a neuron, and it determines whether the final output signal should be sent to the next neuron. When a neuron sends a signal, only the input signal which is greater than a certain threshold will be sent to the next neuron.

3.5 Rectified Linear Unit (ReLU) Function

Currently, ReLU is the most popular activation function used, and it is represented by the following equation:

$$Relu(x) = max\ (0, x) \quad , \tag{16}$$

where the input x is equivalent to a linear function for positive values and zero for negative values. The ReLU function solves the gradient vanishing problem because the slope has a value of 0 or 1. In addition, because this function is non-linear, multiple layers can be used, and this function, which does not use any exponential functions, results in a learning that is approximately six times faster than that achieved by a sigmoid or tanh function.

3.6 Learning of Perceptron

The learning of the perceptron is achieved by changing the values of weight $w = [w_0, w_1, w_2, \cdots, w_d]^T$ such that the output y^t calculated for the input $x^t = [{x_1}^t, {x_2}^t, {x_3}^t, \cdots, {x_d}^t]^T$ is the desired value r^t.[21,23] If the activation function of the output node is linear, the output node is given by the following equation:

$$y^t = \sum_{k=0}^{d} w_k {x_k}^t \tag{17}$$

To minimize the mean-squared error (MSE) of the output node, the weight values are changed using the gradient descent method. This is performed by differentiating the following equation with respect to the weight w_k:

$$E = \frac{1}{2}(r^t - y^t)^2 \tag{18}$$

$$\frac{\partial E}{\partial w_k} = \frac{\partial E}{\partial y^t}\frac{\partial y^t}{\partial w_k} = -(r^t - y^t){x_k}^t \tag{19}$$

The change in weight is thus represented by the following equation:

$$\therefore \Delta w_k = -\eta \frac{\partial E}{\partial w_k} = \eta(r^t - y^t){x_k}^t \tag{20}$$

To solve the classification problem, the output node is assigned a sigmoid activation function. Therefore, for an input x^t, the output y^t is calculated using the following equations:

$$\hat{y}^t = \sum_{k=0}^{d} w_k {x_k}^t \tag{21}$$

$$y^t = f(\hat{y}^t) = \frac{1}{1+e^{-\hat{y}^t}} \tag{22}$$

$$\frac{\partial E}{\partial w_k} = \frac{\partial E}{\partial y^t}\frac{\partial y^t}{\partial \hat{y}^t}\frac{\partial \hat{y}^t}{\partial w_k} = -(r^t - y^t)y^t(1 - y^t){x_k}^t \tag{23}$$

$$\therefore \Delta w_k = -\eta \frac{\partial E}{\partial w_k} = \eta(r^t - y^t)y^t(1 - y^t){x_k}^t \tag{24}$$

Here, the target value r^t is set to 0 or 1, which are the minimum and maximum values of the sigmoid function, respectively. When r^t=1 and y^t=0 or vice versa, the change in weight is 0, thus stopping the learning. In such cases, incorrect saturation occurs in the output node.

This phenomenon can be eliminated by applying the cross-entropy error function to the weight change equation.

$$E_{CE} = -r^t log y^t - (1 - r^t) log(1 - y^t) \tag{25}$$

$$\frac{\partial E_{CE}}{\partial w_k} = \frac{\partial E_{CE}}{\partial y^t}\frac{\partial y^t}{\partial \hat{y}^t}\frac{\partial \hat{y}^t}{\partial w_k} = -(r^t - y^t){x_k}^t \tag{26}$$

$$\therefore \Delta w_k = -\eta \frac{\partial E_{CE}}{\partial w_k} = \eta(r^t - y^t){x_k}^t \tag{27}$$

3.7 Multi-Layer Perceptron (MLP)

An SLP can only solve linearly separable problems because of the functional limitations of a single interface in the input space. Therefore, an MLP that stacks the SLPs, which divide the input space into several interfaces, is used to carry out complex functions. In an MLP, the input, output, and hidden layer node values are represented by the vectors $\boldsymbol{x} = [x_1, x_2, \cdots, x_p]^T$, $\boldsymbol{y} = [y_1, y_2, \cdots, y_m]^T$, and $\boldsymbol{h} = [h_1, h_2, \cdots, h_L]^T$, respectively. The learning pattern is stored in $\boldsymbol{D} = \{(x^p, t^p)\}_{p=1}^{P}$, which is formed by pairing the input and target output vectors. $\boldsymbol{h}$ is termed as the hidden layer because the values in this layer are not provided externally; instead, they are generated during the learning process. A weight matrix $\boldsymbol{W}$ existing between the input and hidden layers transfers the node values from the input layer to the hidden layer, and the weight matrix $\boldsymbol{V}$ existing between the hidden and output layers transfers the node values from the hidden layer to the output layer.

3.8 Error Back-Propagation Learning

In an MLP, when the hidden node vector ***h*** and the output vector ***y*** are calculated for an arbitrary input vector ***x***, and the target vector is defined by ***t***, the distance between the output and target vectors is generally measured using the MSE function.[21]

$$E_{MSE} = \frac{1}{2}\sum_{k=1}^{M}(t_k - y_k)^2 \tag{28}$$

Because the weights are changed by the gradient descent method to minimize this distance, the MSE function is differentiated with respect to the weight, and the weight is changed in the direction of the decreasing derivative.

$$\frac{\partial E_{MSE}}{\partial v_{kj}} = \frac{\partial E_{MSE}}{\partial y_k}\frac{\partial y_k}{\partial \widehat{y_k}}\frac{\partial \widehat{y_k}}{\partial v_{kj}} \tag{29}$$

$$\Delta v_{kj} = -\eta\frac{\partial E_{MSE}}{\partial v_{kj}} \quad , \tag{30}$$

where η is the learning rate.

$$\delta_k \equiv -\frac{\partial E_{MSE}}{\partial y_k} = (t_k - y_k)f'(\widehat{y_k}) \tag{31}$$

If Eq. (31) is used to define the output error signal, the weight change equation is represented as follows:

$$\Delta v_{kj} = \eta(t_k - y_k)f'(\widehat{y_k})h_j \tag{32}$$

The weight change equation of the hidden layer can also be obtained by applying the learning rate in the direction of the decreasing derivative and by finding the derivative of the MSE function with respect to the middle layer weight vector w_{ji}.

$$-\frac{\partial E_{MSE}}{\partial w_{ji}} = -\sum_{k=1}^{M}\frac{1}{2}\frac{\partial(t_k-y_k)^2}{\partial y_k}\frac{\partial y_k}{\partial \widehat{y_k}}\frac{\partial \widehat{y_k}}{\partial h_j}\frac{\partial h_j}{\partial \widehat{h_j}}\frac{\partial \widehat{h_j}}{\partial w_{ji}} = \sum_{k=1}^{M}\delta_k v_{kj}\, f'(\widehat{h_j})x_i \tag{33}$$

Here, we define the error back-propagation from the output layer to the hidden node h_j as follows:

$$\delta_j^{(hidden)} \equiv -\frac{\partial E}{\partial \widehat{h_j}} = f'(\widehat{h_j})\sum_{k=1}^{M} v_{kj}\delta_k \tag{34}$$

The modified weight change equation for the hidden layer weight w_{ji} is obtained as follows:

$$\Delta w_{ji} = \eta x_i f'\left(\widehat{h_j}\right)\sum_{k=1}^{M} v_{kj}\delta_k = \eta\delta_j^{(hidden)}x_i \tag{35}$$

In this equation, when the error signal of each layer is obtained, the weight is changed in such a way that the learning rate is multiplied by the error signal and the lower node value.

3.9 Deep Neural Networks

Neurons in the human nervous system are immensely large in number and are intertwined in extremely complex structures, forming a huge network termed as neural networks.[21,24,25] Similar to this neural network structure, an artificial neural network, which connects MLPs, is devised where each perceptron is considered as the building block.

In the neural network used in this study, the error back-propagation learning is performed using the ReLU function instead of the sigmoid function for the non-linear activation function of the hidden layer, and the softmax function is used as the activation function of the output.

The softmax function and its derivatives are represented by the following equations:

$$y_k = \frac{e^{\widehat{y_k}}}{\sum_j e^{\widehat{y_j}}} \tag{36}$$

$$\frac{\partial y_k}{\partial \widehat{y_j}} = \begin{cases} y_k(1 - y_k) & if \;\; j = k \\ -y_k y_j & otherwise \end{cases} \tag{37}$$

Thus, the output error signal is obtained as follows:

$$\delta_k \equiv -\frac{\partial E_{CE}}{\partial \widehat{y_k}} = (t_k - y_k) \tag{38}$$

3.10 Performance Evaluation of Classifier

The performance of a classifier that has completed supervised learning is evaluated using the classifier accuracy, which is calculated as follows:

$$Accuracy = \frac{No.(Correctly\ Classified\ Samples)}{No.(Presented\ Samples)} \tag{39}$$

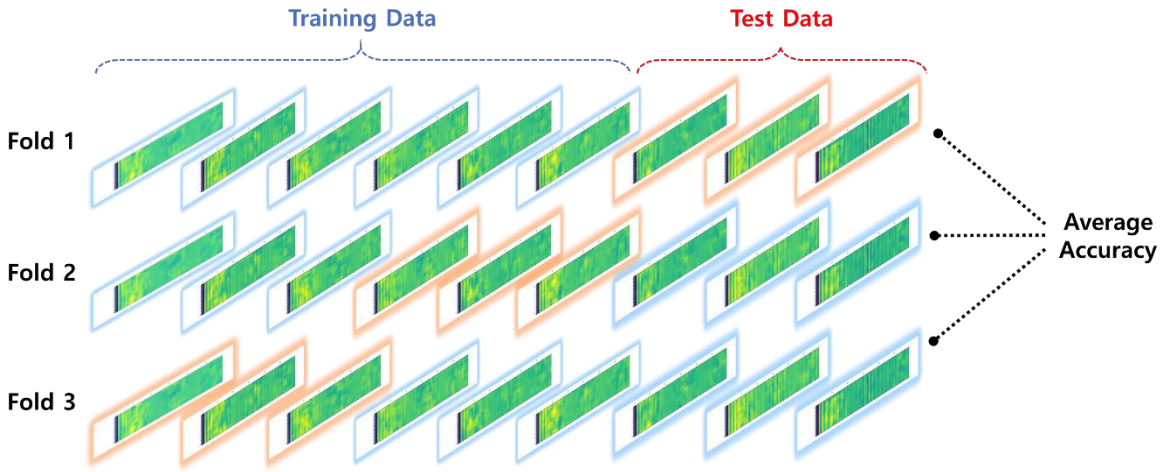
The classification problem has two classes: P (positive) and N (negative). Subsequently, when the sample is input to the classifier, the decision can be divided into four cases, which are collectively called the confusion matrix. A true positive (TP) indicates the number of branches determined by the P class as the actual p class, and a false positive (FP) indicates the number determined by the P class as the actual n class. Similarly, a false negative (FN) indicates the number of branches determined by the N class as the p class, and a true negative (TN) indicates the number of branches determined by the N class as the actual n class.

Therefore, the accuracy is defined as follows:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{40}$$

3.11 K-fold Cross Validation

Cross validation is one of the methods used for evaluating models in statistics or ML. To evaluate the performance of the classifier, a part of the data sample is used for training and the rest is used for the test. The reliability of performance evaluation decreases depending on the number of data samples and classes. In addition, the performance of a classifier can vary depending on the test set configuration, and if this biases the model evaluation indicators, the reliability will be significantly low. To solve this problem, we employ the K-fold cross validation, which ensures that all the data are used at least once in a test set.



Supplementary Figure 3. Cross validation

In fact, the cross validation used in this study divides the data into four groups randomly, extracts each performance indicator, and then averages the performance accuracy in each model.

3.12 Hidden Markov Model (HMM)

The Markov model is a model generally used to analyze any change in phenomena such as weather or stock prices. The HMM extends this Markov model by adding hidden states and direct observations. The problem we are trying to solve using HMM is that only y_t is observable, and because y_t is dependent on q_t, the sequences of q_t can be inferred from the sequence of y_t.

Since the 1970s, HMM has been actively applied in the field of speech recognition, and it is currently used for recognizing patterns in sequential data affected by previous data such as handwriting and motion recognition.

The first step is to calculate the probability $p(O|\theta)$ that a given observation $O = o_1, o_2, \cdots, o_T$ will be observed when all the parameters of the HMM have been determined. The probability of the first observation o_1 is as follows:

$$p(o_1|\theta) = \sum_{i_1} p(q_{i_1}) p(o_1|\, q_{i_1}) \tag{41}$$

Therefore, the probability of the observation $\mathrm{O} = o_1, o_2, \cdots, o_T$ until time T is as follows:

$$p(\mathrm{O}|\theta) = \sum_{i_1} \sum_{i_2} \cdots \sum_{i_T} p(q_{i_1}) p(o_1|q_{i_1}) p(q_{i_2}|q_{i_1}) p(o_2|q_{i_2}) \cdots p(q_{i_T}|q_{i_{T-1}}) p(o_T|q_{i_T}) \tag{42}$$

3.13 Convolutional Neural Network (CNN)

The CNN is composed of a building block consisting of an input layer, a convolution layer, pooling layer, and fully connected layer. In the CNN, one node of the convolution layer calculates the sum of weights by multiplying the inputs of digital filter coefficients with the subtractive region that acts only on a certain area of the input and then adding the results. If there are H convolution nodes (here, H is called the depth), then H vectors with I elements are obtained for the input layer, and are called feature maps. If there are z elements in the weight vector of a convolution node, the convolution operation is calculated as follows:

$$h_j[i] = w_j * x = \sum_{i=-(z-1)/2}^{(z-1)/2} w_j[k] x[i+k] \tag{43}$$

In the 2D representation of the input x and filter, the convolution operation must be calculated by overlaying the filter on the input plane in the form of a plane:

$$h_j[u,v] = w_j * x = \sum_{i=-(z-1)/2}^{(z-1)/2} \sum_{k=-(z-1)/2}^{(z-1)/2} w_j[i,k] x[u+i, v+k] \tag{44}$$

When performing a convolution operation with the filter function, we cannot place the center of the convolution filter on the edge. To prevent this phenomenon, convolution is performed by adding a specific value to the edge such that the center of the convolution filter is never located at the edge; this is called padding.

Padding uses a value of zero or a value of symmetry at the input. In addition, the filter function calculates the convolution results by varying the travel relative to the input. In this study, for each element, the feature maps obtained in the convolution layer are sent through an activation function before entering the pooling layer, and the ReLU function is used to enhance the non-linear features without affecting the subtractive area of the convolutional layer. The final layer of the CNN consists of fully connected layers, and the softmax activation function is used to calculate the sum of weights input to the output nodes.

3.14 Pooling layer

The pooling layer of the CNN also plays an important role. Non-linear down-sampling is performed using H vectors with I elements obtained from the convolution layer as inputs to the pooling layer. In this study, we use max-pooling, which divides the input into several non-overlapping areas and

performs pooling. This approach considers that the approximate relative relationship with other features is important rather than the exact location of the features extracted by the convolution filter. In addition, there are three advantages of using this approach. First, the down-sampling with pooling reduces the representation area of the feature, thereby decreasing the number of parameters and the amount of computation. Second, it prevents excessive overfitting, and third, it can be made robust against the travel and distortion, which we aim to analyze.

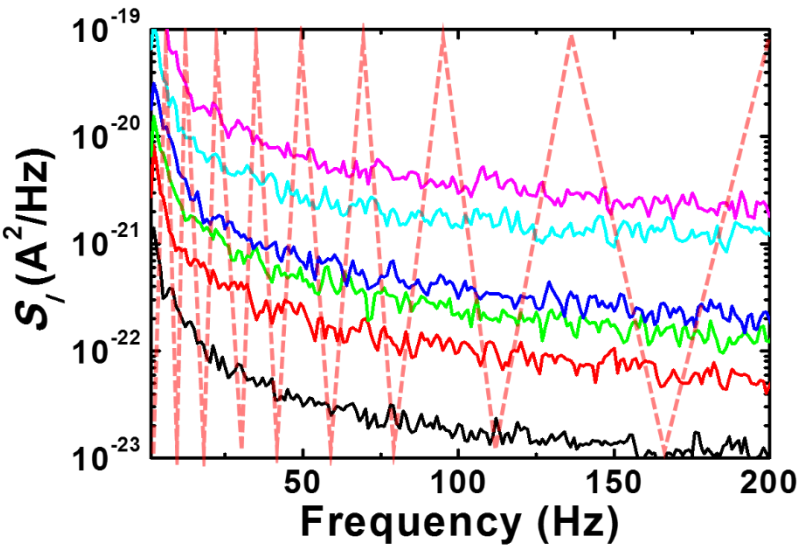
3.15 Mel Frequency Cepstral Coefficient (MFCC)

MFCC is one of the most widely used algorithms in speech recognition. To recognize speech, actual sound features must be extracted from the noise and background sounds in the input signal. Therefore, instead of targeting the entire input sound, the MFCC only extracts the required sound features by analyzing the spectra of frames.

In this study, the input signals are used to extract the features of carrier behavior from the very small fluctuations of current generated when a certain voltage is applied, instead of the sound signal. The extraction of features using MFCC can be divided into six steps.

In the first step, the signal in the time domain is divided into small-sized frames. The input data are measured for a duration of 1 s with a sampling rate of 20000–33333 Hz at intervals of 33–50 μs, and the frame size is 10 ms.

In the second step, the power spectrum of each frame is obtained and the periodogram for estimating power spectral densities is calculated. Assuming that the input signal is a sound signal, each cochlear nerve transfers the input frequency to the brain, depending on where the cochlea vibrates. In MFCC, this periodogram acts similar to that of the human hearing system. Here, the power spectrum calculated in each frame can be observed as the power spectrum of the current, and we can obtain the energy pattern distributed over various frequencies. Therefore, the power spectral estimation of carrier behavior is performed using the power spectra of frames in the periodogram, and the period of trapping/detrapping and scattering phenomena of the carrier can be estimated. The periodic power bins are formed by the current power spectrum, and the groups can be added to show the distribution of energy according to the carrier behavior over various frequencies. In the third step, a Mel scale filter is applied to the power spectrum obtained from each frame, indicating the amount of energy present around the low frequencies. When the Mel scale filter is applied, the width of the filter increases with the frequency, and it is hardly considered at high frequencies.



Supplementary Figure 4. Mel scale filter on power spectrum

$$f = 2595\log\left(1 + \frac{f}{700}\right) \tag{45}$$

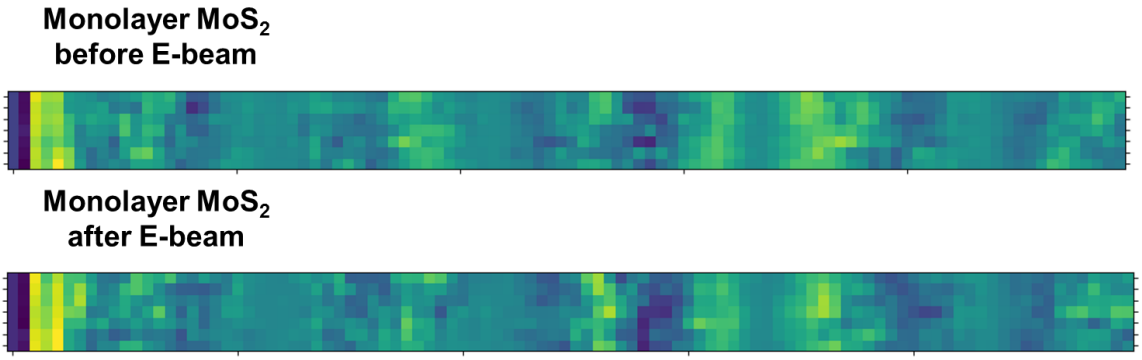
The use of the Mel scale filter increases the tendency of carriers to be driven toward deeper traps operating at low-frequencies than that toward shallower traps operating in high frequency regions.

In the fourth step, the log of the energy applied to the Mel filter is obtained. If the overall current fluctuations are severe, the differences in energy change are insignificant.

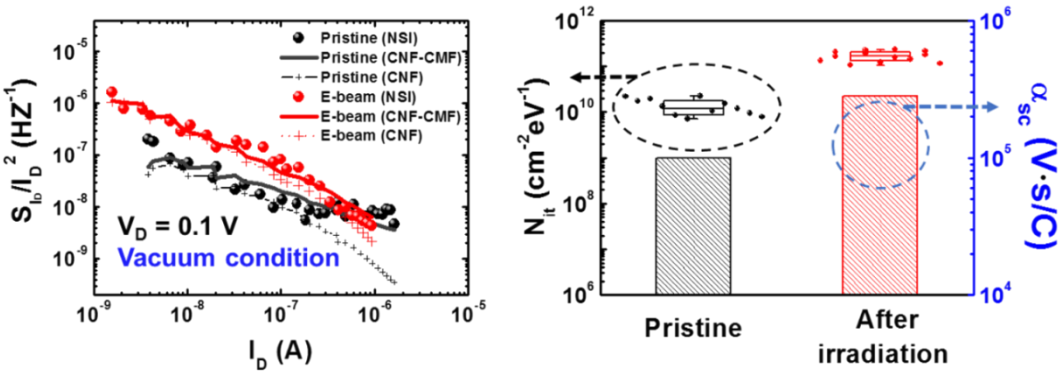
In the fifth step, the discrete cosine transform (DCT) of the logarithmic energy spectrum is calculated, the correlation between the energies of Mel filter is separated, and the diagonal covariance matrix is used.

$$c_n = \sum_{k=1}^{K} \log\big(E(k)\big) \cos\left[n\left(k - \frac{1}{2}\right)\frac{\pi}{M}\right], \quad for\ n = 1, \cdots, K \tag{46}$$

In the final step, some MFCC coefficients calculated by the DCT are extracted to form a 2D array.



Supplementary Figure 5. Featured 2D array of monolayer MoS_2 before and after e-beam irradiation



Supplementary Figure 6. Normalized power spectral densities and extracted N_{it} and α_{sc} of monolayer MoS_2 before and after e-beam irradiation.

The featured 2D array calculated by MFCC exhibits a visible change, but classifying this extensive data accurately is considerably difficult. In particular, when the monolayer MoS_2 is fitted using low-frequency noise analysis, the interface trap density and scattering parameter increase significantly after the e-beam irradiation, but analyzing this change clearly using the featured 2D array is challenging.

3.16 Gaussian mixture model (GMM)

Gaussian mixture model (GMM) is the algorithm to find the most optimized representation of the data. Specifically, assuming that the data consist of k Gaussians, GMM is an algorithm that finds k averages and covariances that best describe the data.[26] Every machine learning problem has a performance measure, and the performance measure of GMM is a log likelihood function, which aims to find the parameter that provides the largest probability of data X for a given parameter. The log likelihood is defined as $\ln p(X|\theta)$, and the parameters to be determined comprise each Gaussian mean μ_j, covariance Σ_j, and finally the probability π_j that each datum belongs to the corresponding Gaussian. Therefore, if we define $N(x_i|\mu_i, \Sigma_i)$ as the multinomial Gaussian distribution of x_i for μ_j and Σ_j, the log likelihood function is represented as follows:

$$\ln p(X|\pi, \mu, \Sigma) = \sum_i^n \ln \sum_j^k \pi_j N(x_i|\mu_i, \Sigma_i) \tag{47}$$

Among the algorithms used to solve the GMM, each step is solved using the EM algorithm. In the E-step, the latent variable with the highest expectation is found by using the μ, Σ, and π values currently provided. In the M-step, we use the newly estimated latent variable to find μ, Σ, and π that maximize that value. The EM algorithm iterates between the E-step and M-step until the value is not changed further.

3.17 Available source code at Github

MFCCs and filter bank energies

https://github.com/jameslyons/python_speech_features

HMM learning methods

https://github.com/hmmlearn/hmmlearn

GMM learning methods

https://github.com/stober/gmm

CNN image classifier

https://github.com/gsurma/image_classifier

NN deep learning methods

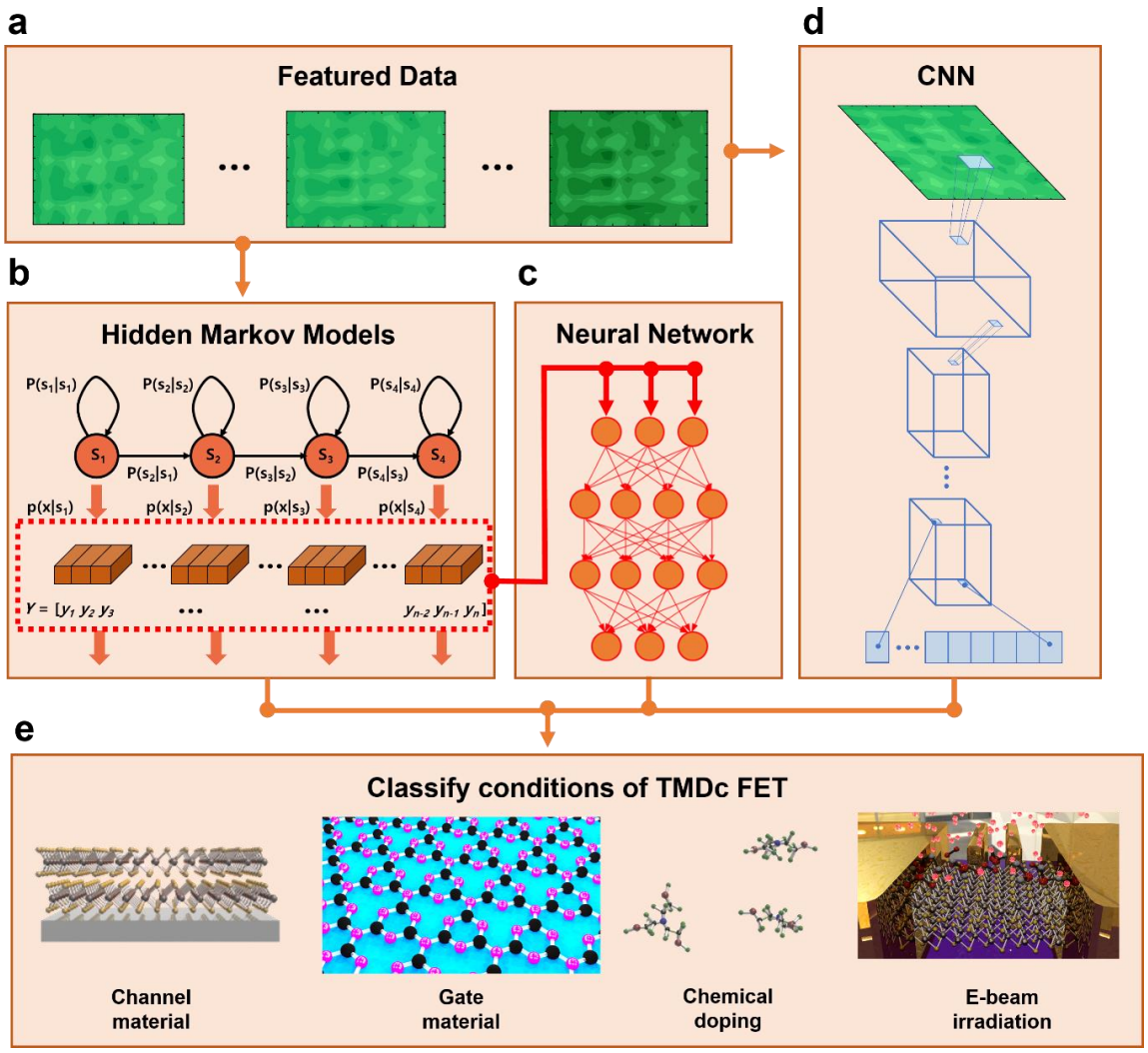
https://github.com/mnielsen/neural-networks-and-deep-learning

K-fold cross validation methods

https://github.com/encodedANAND/K-Fold-Cross-Validation

Supplementary Note 4: Additional learning algorithm with CNN and GMM

4.1 Flowchart of additional learning algorithm with CNN



Supplementary Figure 7. ML and DL processes used to classify devices operated under various conditions: (a) Classifying the current noise data of various devices into 2D arrays; **(b)** learning the 2D arrays using classes, and then calculating the scores using HMMs to arrange device classes; **(c)** classifying the classes by learning each score extracted using the HMM through NNs; **(d)** learning and classifying the data featured in 2D arrays using CNNs; and **(e)** classifying the devices based on various conditions including channel materials, gate materials, chemical doping, and e-beam irradiation using the three aforementioned learning methods.

Based on the work performed thus far, the engineered current MFCC features—which had been characterized into 2D arrays with a number between 200 and 1000 for each device class[3,4,6–9,12,14–16]—are displayed as shown in Supplementary Figure 7. All the data features composed into 2D arrays

exhibited their own unique characteristics, which depended on the gate voltage range, channel material, contact metal, and gate dielectric.

Supplementary Figure 7b and 7c present the learning methods using the HMM and NN algorithms mentioned in the manuscript, and additional algorithm is the learning method using a CNN algorithm, as shown in Supplementary Figure 7d.

The additional method, which used the CNN algorithm, was employed to categorize the classes by extracting characteristics from the engineered MFCC modulus, as shown in Supplementary Figure 7a, as 3D data, such as a color photograph, without learning through either the HMM or NN. Using the CNN,[23,25,27] we entered the featured MFCC data as the input image data, and obtained feature maps as outputs by iterating convolution calculations (through the kernel) using the weight parameter (Supplementary Figure 7d). The feature map was transferred to the next layer after performing a max-pooling with the window size (2,2) on it. This process was repeated for all the layers. The characteristics calculated by pooling were less affected by the variation in position at the origin and featured data to determine the invariance and overcome the spatial change. In addition, max-pooling,[21,28] which found the maximum value of the region, was more efficient than average pooling, which used the average value of the region. The flattened layer was extracted by changing the shape of the max-pooling layer, i.e., converting it into a vector, and the probability density function for each class was obtained as the output, using the ReLU and softmax functions as activation functions. Finally, based on the extracted probability density function of each class, we classified the device classes, as shown in Supplementary Figure 7e. Learning was performed with a convolution neural network (CNN) to achieve a classification accuracy of 94.77 %.

4.2 Additional learning algorithm with GMM

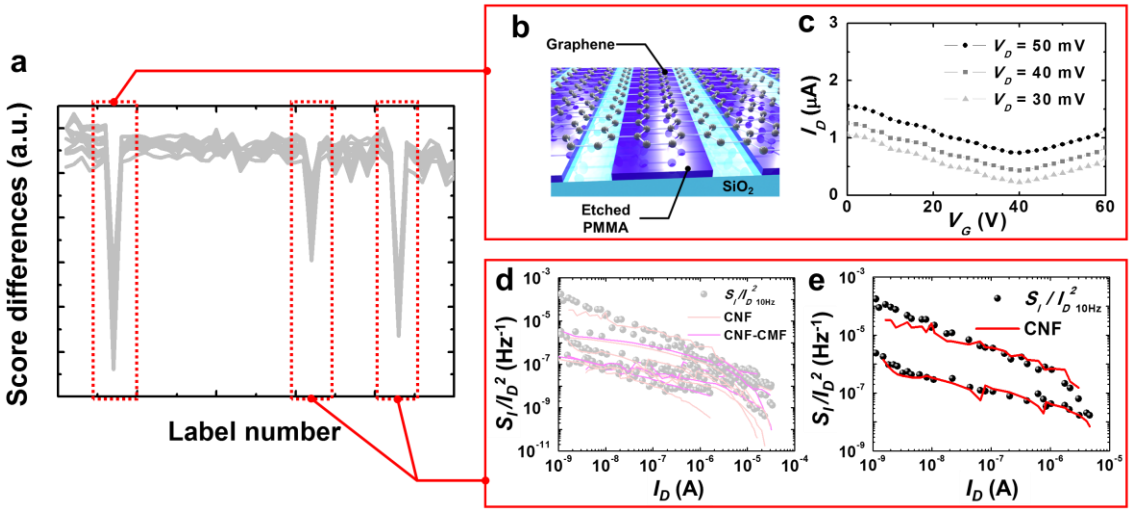
In this additional method, we used a Gaussian mixture model (GMM)[26,29] with 10 components, instead of the HMM, for carrier behavior analysis, and classified the 2D arrays consisting of cepstral coefficients into each class. The computational requirements of the GMM approach used in the analysis for current fluctuations were considerably lower than those of the MLP approach. In addition, the dynamic characteristics of the current signal fluctuations caused by trapping/detrapping, or by Coulomb scattering, indicated that the GMM method was well-suited for device classification. The 10-component GMM equation (Eq. (2)) was used to train the data feature and develop the output distribution.[21,26,29]

$$b_j(x) = p(x|S = j) = \sum_{m=1}^{10} c_{jm} \mathcal{N}(x; \mu_{jm}, \Sigma_{jm}) \tag{48}$$

Based on the GMM, we defined a variable to store the model for each data feature and defined both a model learning function and a function to calculate the model score vector, *Y*, for the input data feature. Subsequently, we trained the model using this data feature and saved it. Afterwards, we selected the

model with the highest score from the test data set, and then obtained the classification results. In the second additional algorithm, learning was performed using a GMM showing a classification accuracy of 94.5 %.

Supplementary Note 5: Score differences analysis



Supplementary Figure 8. Score differences analysis: (a) relative HMM score differences extracted from each class; **(b)** illustration of graphene FETs; **(c)** transfer characteristics (I_D-V_G) of graphene FETs; **(d)** variations in the measured normalized drain current spectral densities for each class at f = 10 Hz and data fitted to the CNF and CNF-CMF models; and **(e)** variations in the measured normalized drain current spectral densities for WSe_2 FETs, using Ti as the contact metal, at f = 10 Hz and data fitted to the CNF model.

Supplementary Figure 8a shows the score differences between the corresponding and other labels extracted using the HMM method for each class. The score differences between the labels were a distinguishing measure, with three specific labels exhibiting extremely large differences. The label of the graphene FET exhibited the largest score difference, as shown in Supplementary Figure 8b. When analyzing the low-frequency noise of graphene devices, we observed that they followed the *1/f* trends; however, contrary to the other 2D nanosheet FETs analyzed in this study, they dwere not fitted to either the CNF or CNF-CMF models, and they exhibited the characteristics of conductors.

The transfer characteristics (I_D-V_G) of a graphene FET at drain voltages of 30, 40, and 50 mV are shown in Supplementary Figure 8c. The graphene FET did not turn off completely, and the current on/off ratio was not even 1, when the current was increased at higher V_D values according to the V_G variation. Consequently, the graphene FETs were observed to be perfectly classified with the other 2D layered FETs, when considered in relation to other labels.

The other two labels that exhibited high score differences were of the WSe_2 FETs, which used Ti as their source/drain contact metal, and they were fitted to the CNF model and not to the CNF-CMF model. Supplementary Figure 8d shows the normalized drain current spectral densities, S_I/I_D^2, at f = 10 Hz for

all the devices, except for the graphene devices. Most of the devices fitted well to the CNF-CMF model, which can be attributed to their mobility fluctuations. Supplementary Figure 8e shows the normalized drain current spectral densities at 10 Hz for WSe_2 FETs that used Ti as the contact metal, and they fitted well to the CNF model, without any Coulomb scattering, in contrast to the other 2D nanosheet FETs.

The charge carriers in a semiconductor, such as MoS_2 and WSe_2, are scattered by lattice vibrations and impurities, such as dopants and defects. Thus, the carrier velocities change drastically with the changes in energy and momentum. The scattering parameter in an n-region can be expressed as shown in Eq. (49):[20]

$$\alpha_{SC} = \frac{1}{\mu_{eff}^2}\frac{\partial \mu_{eff}}{\partial Q_{ox}} = \frac{1}{\mu_{ac}^2}\frac{\partial \mu_{ac}}{\partial Q_i} + \frac{1}{\mu_{sr}^2}\frac{\partial \mu_{sr}}{\partial Q_i} + \frac{1}{\mu_{C,imp}^2}\frac{\partial \mu_{C,imp}}{\partial Q_i} + \frac{1}{\mu_{C,ox}^2}\frac{\partial \mu_{C,ox}}{\partial Q_i}, \quad (49)$$

where μ_{eff} represents the channel effective mobility, μ_{ac} is the mobility limited by surface acoustic phonon scattering, μ_{sr} is the mobility caused by surface roughness scattering, $\mu_{C,imp}$ represents the mobility caused by Coulomb scattering occurring mainly due to ionized impurities, and $\mu_{C,ox}$ is the mobility limited by Coulomb scattering mainly caused by the fixed/trapped charge in the gate oxide.

The first two terms on the right hand side decrease with the increasing Q_i, whereas the third term increases; these three terms are often neglected in the derivation of α_{SC}.

As shown in Supplementary Figure 8e, the contact metals, such as Ti and Ag, used in WSe_2 FETs have low work functions, which are close to the electron affinity of WSe_2 channel; this results in a small Schottky barrier.[15,30–33] Thus, the last term in Eq. (49) can be approximated, and Eq. (50) can be obtained:

$$\mu_{C,ox} = \frac{1}{\alpha_C |Q_{ox}|}, \ \alpha_{SC} \approx \alpha_C \quad (50)$$

Because the inversion electron charge interrupts the Coulomb interaction between the oxide charge and carrier in the inversion layer, the magnitude of α_C decreases with increasing Q_i. As a result, FETs that did not fit with the CNF-CMF model had significantly high score differences, and they were well classified similar to the graphene FETs.

Supplementary References

1. Ji, H. *et al.* Suppression of Interfacial Current Fluctuation in $MoTe_2$ Transistors with Different Dielectrics. *ACS Appl. Mater. Interfaces* **8**, 19092–19099 (2016).

2. Joo, M. K. *et al.* Understanding Coulomb Scattering Mechanism in Monolayer MoS2 Channel in the Presence of h-BN Buffer Layer. *ACS Appl. Mater. Interfaces* **9**, 5006–5013 (2017).

3. Na, J. *et al.* Few-layer black phosphorus field-effect transistors with reduced current fluctuation. *ACS Nano* **8**, 11753–11762 (2014).

4. Ko, S. P. *et al.* Low frequency noise reduction in multilayer WSe_2 field effect transistors. *IEEE-NANO 2015 - 15th Int. Conf. Nanotechnol.* 1118–1121 (2015).

5. Joo, M. *et al.* Electron Excess Doping and Effective Schottky Barrier Reduction on the MoS_2/h-BN Heterostructure. *Nano Lett*. **16**, 6383-6389 (2016).

6. Na, J. *et al.* Low-frequency noise in multilayer MoS_2 field-effect transistors: the effect of high-k passivation. *Nanoscale* **6**, 433–441 (2014).

7. Ryu, M. Y. *et al.* Triethanolamine doped multilayer MoS_2 field effect transistors. *Phys. Chem. Chem. Phys.* **19**, 13133–13139 (2017).

8. Cho, Y. H. *et al.* Soft-type trap-induced degradation of MoS 2 field effect transistors. *Nanotechnology* **29**, 1–8 (2018).

9. Jin, J. E. *et al.* Catalytic etching of monolayer graphene at low temperature via carbon oxidation. *Phys. Chem. Chem. Phys.* **18**, 101–109 (2016).

10. Jin, J. E. *et al.* Surface Modulation of Graphene Field Effect Transistors on Periodic Trench Structure. *ACS Appl. Mater. Interfaces* **8**, 18513–18518 (2016).

11. Na, J. *et al.* Low-frequency noise in multilayer MoS2field-effect transistors: the effect of high-k passivation. *Nanoscale* **6**, 433–441 (2014).

12. Lee, B. C. *et al.* Probing Distinctive Electron Conduction in Multilayer Rhenium Disulfide. *Adv. Mater.* **31**, 1–7 (2019).

13. Ji, H. *et al.* Thickness-dependent carrier mobility of ambipolar $MoTe_2$: Interplay between interface trap and Coulomb scattering. *Appl. Phys. Lett.* **110**, 183501 (2017).

14. Ko, S. P. *et al.* Current fluctuation of electron and hole carriers in multilayer WSe_2 field effect transistors. *Appl. Phys. Lett.* **107**, 242102 (2015).

15. Ko, S. *et al.* Few-Layer WSe2 Schottky Junction-Based Photovoltaic Devices through Site-

Selective Dual Doping. *ACS Appl. Mater. Interfaces* **9**, 42912–42918 (2017).

16. Ko, S. P. *et al.* Capacitance-voltage analysis of electrical properties for WSe_2 field effect transistors with high-k encapsulation layer. *Nanotechnology* **29**, 065703 (2018).

17. Joo, M. K. *et al.* A dual analyzer for real-time impedance and noise spectroscopy of nanoscale devices. *Rev. Sci. Instrum.* **82**, 034702 (2011).

18. Kirton, M. J. & Uren, M. J. Noise in solid-state microstructures: a new perspective on individual defects, interface states and low-frequency (1/f) noise. *Adv. Phys.* **38**, 367–468 (1989).

19. Schroder, D. K. *Semiconductor Material and Device Characterization* (John Wiley & Sons, 2006).

20. Von Haartman, M. & Östling, M. *Low-Frequency Noise in Advanced MOS Devices (*Springer Science & Business Media, 2007*)*

21. Borovcnik, M., Bentz, H.-J. & Kapadia, R. *A Probabilistic Perspective*. *Chance Encounters: Probability in Education* (Springer, Dordrecht, 1991).

22. Ghahramani, Z. Probabilistic machine learning and artificial intelligence. *Nature* **521**, 452–459 (2015).

23. Lawrence, S., Giles, C. L., Tsoi, A. C. & Back, A. D. Face recognition: A convolutional neural-network approach. *IEEE Trans. Neural Networks* **8**, 98–113 (1997).

24. Hara, K., Saito, D. & Shouno, H. Analysis of function of rectified linear unit used in deep learning. *Proc. Int. Jt. Conf. Neural Networks* 1–8 (2015).

25. Cireşan, D. C., Meier, U., Gambardella, L. M. & Schmidhuber, J. Convolutional neural network committees for handwritten character classification. *Proc. Int. Conf. Doc. Anal. Recognition, ICDAR* **10**, 1135–1139 (2011).

26. Shao, X. & Milner, B. MAP prediction of pitch from mfcc vectors for speech reconstruction. *8th Int. Conf. Spok. Lang. Process. ICSLP 2004* **1**, 2425–2428 (2004).

27. Kim, Y. Convolutional neural networks for sentence classification. *EMNLP 2014 - 2014 Conf. Empir. Methods Nat. Lang. Process. Proc. Conf.* 1746–1751 (2014).

28. Masci, J., Giusti, A., Ciresan, D., Fricout, G. & Schmidhuber, J. A fast learning algorithm for image segmentation with max-pooling convolutional networks. *2013 IEEE Int. Conf. Image Process. ICIP 2013 - Proc.* 2713–2717 (2013).

29. Schuller, B. *et al.* Hidden Markov model-based speech emotion recognition. *Proc. - IEEE Int. Conf. Multimed. Expo* **1**, I401–I404 (2003).

30. Kang, W. M. *et al.* Multi-layer WSe_2field effect transistor with improved carrier-injection contact by using oxygen plasma treatment. *Solid. State. Electron.* **140**, 2–7 (2018).

31. Zhang, W. *et al.* Role of metal contacts in high-performance phototransistors based on WSe_2 monolayers. *ACS Nano* **8**, 8653–8661 (2014).

32. Fang, H. *et al.* High-performance single layered WSe2p-FETs with chemically doped contacts. *Nano Lett.* **12**, 3788–3792 (2012).

33. Wang, Y. *et al.* Does p-type ohmic contact exist in WSe2-metal interfaces? *Nanoscale* **8**, 1179–1191 (2016).